4,222 Introduction to Programming

Week 8: The FRED API, Raising Errors, & Input Validation

AUTHOR
Franziska Bender

PUBLISHED
April 21, 2026

```
import pandas as pd
import matplotlib.pyplot as plt
```

Accessing FRED Data

The Federal Reserve Economic Data (FRED) database is one of the most valuable resources for economists. In this section, we will learn how to bypass manual CSV or Excel downloads and use the FRED API to pull economic data directly into our Python environment.

Getting Started

Step 1: Get Your FRED API Key

Before you can write any code, you need permission to access their database.

1. Go to the FRED API website
2. Create a free user account (or log in if you already have one).
3. Navigate to **"API Keys"** in your account dashboard and click **"Request API Key"**.
4. Give it a quick description (e.g., "Python Data Analysis") and generate the key. Copy this 32-character string; you'll need it in Step 3.

Step 2: Install the Required Python Libraries

We are going to use the open-source `fredapi` wrapper. We also need `python-dotenv` for step 3.

1. Open your terminal
2. Run the following command: `uv add fredapi python-dotenv`

Step 3: Secure Your API Key

It is a bad practice to paste your API key directly into your Python script, especially if you plan to share your code or push it to GitHub. Use a `.env` file instead.

A `.env` (environment) file is a simple text file used to safely store secret passwords and API keys outside of your main Python code. Here is how to set it up and ensure it never gets uploaded to the internet:

1. Create a new file named exactly `.env` (don't forget the dot at the beginning) in the root directory of your project.
2. Inside that file, add your API key like this: `FRED_API_KEY=paste_your_32_character_api_key_here`
3. Save and close the `.env` file.
4. Inside the `.gitignore` file, simply type `.env`
5. Save and close the `.gitignore`

Why did we do this? By adding `.env` to your `.gitignore`, you are instructing Git and GitHub to pretend the file doesn't exist. When you eventually share your project, your Python code goes online, but your secret key stays safely hidden on your own computer.

Retrieving data from FRED

With the setup complete, we can now write the code to connect to FRED and retrieve data.

1. Load the Key and Initialize the Connection

First we need to import our required tools and tell Python how to find the secret key we hid in the `.env` file. Then we can initialize the FRED API connection:

```
# Import required packages
import os                                                    1
from dotenv import load_dotenv
from fredapi import Fred
import time                                                  2

# 1. Look for the .env file and load the hidden variables inside it
load_dotenv()

# 2. Securely fetch the key using os.getenv()
api_key = os.getenv('FRED_API_KEY')

# 3. Initialize the FRED API connection
fred = Fred(api_key=api_key)                                 3
```

---

2. Retreive a Data Series

Now that we have the keys to the database, let's actually download some data!

FRED organizes its massive database using Series IDs. Every single economic indicator has a unique, short code. For example:

- Gross domestic product is `GDP`
- Consumer Price Index (Inflation) is `CPIAUCSL`
- The national unemployment rate is `UNRATE`

Let's pull the historical data for the Unemployment Rate. The `fredapi` library makes this incredibly simple with a single command: `get_series()`.

Run this code in your next cell:

```
# Fetch the Unemployment Rate data
unemployment_data = fred.get_series('UNRATE')

# Display the first 5 rows of the data
unemployment_data.head()
```

```
1948-01-01    3.4
1948-02-01    3.8
1948-03-01    4.0
```

```
1948-04-01    3.9
1948-05-01    3.5
dtype: float64
```

When you run this code, Python reaches out to the FRED servers, grabs the monthly unemployment data, and downloads it directly into your computer's memory.

Note that the data is saved as a `pandas.Series`, with a `DatetimeIndex`:

```
print(type(unemployment_data))
print(type(unemployment_data.index))
```

```
<class 'pandas.Series'>
<class 'pandas.DatetimeIndex'>
```

---

3. Retrieving Multiple Series

Often, you will want to compare different economic indicators side-by-side. Let's compare the state-level unemployment rates for California, Texas, New York, and Florida.

In FRED, state unemployment rates follow a standard naming convention: the two-letter state abbreviation followed by UR (CAUR, TXUR, etc.).

Instead of writing the same download command four times, we will put our IDs in a list, loop through them, and use the pandas library to concatenate (combine) them into a single DataFrame.

Run this code in your next cell:

```
# 1. Define a list of the exact FRED Series IDs you want
series_ids = ['UNRATE', 'CAUR', 'TXUR', 'NYUR', 'FLUR']

# 2. Create an empty list to store our downloaded pandas Series
series_list = []

# 3. Loop through the list of IDs
for series_id in series_ids:
    print(f"Downloading data for {series_id}...")

    # Fetch the data
    s = fred.get_series(series_id)

    # Name the series so the final DataFrame knows what to call the column
    s.name = series_id                                                    1

    # Append the processed series to our list
    series_list.append(s)

    # Pause for half a second before starting the next iteration
    time.sleep(0.5)                                                       2

# 4. Concatenate all series in the list into a single DataFrame
df_unemployment = pd.concat(series_list, axis=1)                          3
```

```
# 5. Display the most recent 5 rows using .tail()
print("\nData successfully combined!")
df_unemployment.tail()
```

```
Downloading data for UNRATE...
Downloading data for CAUR...
Downloading data for TXUR...
Downloading data for NYUR...
Downloading data for FLUR...

Data successfully combined!
```

	UNRATE	CAUR	TXUR	NYUR	FLUR
2025-11-01	4.5	5.5	4.3	4.6	4.3
2025-12-01	4.4	5.5	4.3	4.6	4.3
2026-01-01	4.3	5.4	4.3	4.6	4.5
2026-02-01	4.4	NaN	NaN	NaN	NaN
2026-03-01	4.3	NaN	NaN	NaN	NaN

How does `pd.concat()` work?

The `pd.concat()` function is a core tool in the pandas library used to combine multiple Series or DataFrames together. The word "concat" is short for concatenation, which simply means linking items together in a sequence.

When you pass a list of data to `pd.concat()`, you have two main parameters that control how the data is stitched together:

1. The Direction (`axis`):
   - `axis=0` (Vertical Stacking): The default. It stacks datasets vertically, placing the second dataset below the first to create longer columns.
   - `axis=1` (Horizontal Alignment): This is what we used. It places the datasets side-by-side to build a wider table with more columns.
2. Index Alignment (`join`): When combining side-by-side (`axis=1`), `pd.concat()` looks at the row labels (the index) to align the data. But what happens if the indexes don't perfectly match? (For example, what if California's data starts in 1976, but Florida's doesn't start until 1980?). This is controlled by the optional `join` parameter, which is set to `'outer'` by default.
   - `join='outer'` (The Default): This tells pandas to take the union of the indexes. It will keep every unique row label it finds across all the series. Our DataFrame will start in 1976, and for the years Florida has no data, pandas automatically fills the empty space with `NaN` (Not a Number) to ensure the rows stay perfectly aligned.
   - `join='inner'` (The Alternative): If you only want to look at the exact overlapping data, you can explicitly set `join='inner'`. This takes the intersection of the indexes, keeping only the rows where all series have data. In our example, it would force the DataFrame to start in 1980, dropping California's older data entirely so that there are no `NaN` values.

Stick to the default `join='outer'` when downloading. This ensures you keep all the data and don't accidentally drop any data upfront. You can decide later on how to deal with missing values, depending on what you need for your analysis.

Handling mixed frequencies

You might want to retrieve series that are reported at different frequencies (for example monthly unemployment rate and quarterly GDP). If you combine mixed-frequency data using `pd.concat()` exactly as we did above, the default `join='outer'` setting will align everything by the exact date. Because it keeps every unique date it finds, your resulting DataFrame will expand to accommodate the highest frequency. For example, you will get a row for every single month, but your quarterly GDP column will only have a value in one month per year, and pandas will fill the other months with `NaN` (Not a Number).

To avoid a messy DataFrame filled with missing values, you can group your series IDs by their reporting frequency beforehand and load them into separate DataFrames (e.g., creating one `df_monthly` and one `df_annual`), instead of loading them all into one.

API Rate Limiting

When you request data using an API, you are asking the provider's servers to perform work. To prevent any single user from overwhelming their systems, organizations usually enforce "rate limits." For example, the FRED API strictly restricts users to a maximum of 120 requests per minute.

If you run a loop that attempts to download hundreds of series at once, Python will execute those requests as fast as your computer allows. When the FRED server detects that you are asking for data too quickly and have exceeded the allowed limit, it will temporarily block your connection and return an error (specifically, an HTTP 429 "Too Many Requests" error), causing your script to crash.

To prevent this, you must explicitly slow down your code. You can achieve this using Python's built-in `time` module. By adding a `time.sleep()` command inside your loop, you instruct Python to pause execution for a fraction of a second before making the next request, ensuring you stay safely under the server's speed limit.

---

4. Retrieving Series Metadata

When you download a series using `get_series()`, you only get the dates and the numbers. But what if you forget the exact definition of the data? Is the GDP measured in millions or billions? Is the data seasonally adjusted?

To find this out, you can use the `get_series_info()` method. This method returns the official metadata from the FRED database (as a pandas Series).

Run this code in your next cell to see the details for the Unemployment Rate:

```
# Fetch the metadata for the UNRATE series
unrate_info = fred.get_series_info('UNRATE')

# Display the information
print(unrate_info)
```

```
id                                                     UNRATE
realtime_start                                     2026-04-03
realtime_end                                       2026-04-03
title                                    Unemployment Rate
observation_start                                  1948-01-01
observation_end                                    2026-03-01
frequency                                             Monthly
frequency_short                                             M
```

```
units                                                          Percent
units_short                                                          %
seasonal_adjustment                                   Seasonally Adjusted
seasonal_adjustment_short                                           SA
last_updated                                        2026-04-03 08:19:15-05
popularity                                                          98
notes                        The unemployment rate represents the number of...
dtype: str
```

Here are the most useful pieces of information you will find in that output:

- id: the series ID
- title: The official, full name of the series (e.g., “Unemployment Rate”).
- units: How the data is measured (e.g., “Percent” or “Billions of Dollars”).
- frequency: How often the data is recorded (e.g., “Monthly” or “Annual”).
- seasonal_adjustment: Whether the data has been mathematically smoothed out to account for predictable seasonal changes (like holiday hiring spikes).
- notes: Detailed information on the variable (not all series have this information, e.g. UNRATE does but NYUR does not)

```
print(unrate_info['notes'])
```

```
The unemployment rate represents the number of unemployed as a percentage of the labor force.
Labor force data are restricted to people 16 years of age and older, who currently reside in 1
of the 50 states or the District of Columbia, who do not reside in institutions (e.g., penal
and mental facilities, homes for the aged), and who are not on active duty in the Armed
Forces.  This rate is also defined as the U-3 measure of labor underutilization.  The series
comes from the 'Current Population Survey (Household Survey)'  The source code is: LNS14000000
```

---

5. Retrieving Metadata for Multiple Series

Just like we downloaded the actual numbers for multiple states at once, we can also download the metadata for multiple series at once and organize it into a clean table.

When working with many different data sources, it is incredibly helpful to create a “data dictionary”—a summary table that tells you exactly what each column in your main dataset represents.

To do this, we will loop through our list of series IDs again. But this time, instead of pulling the numbers, we will pull the info, extract only the specific details we care about (like the title and frequency), and compile them into a new DataFrame.

```
import pandas as pd

# 1. Define the list of exact FRED Series IDs we are working with
series_ids = ['UNRATE','CAUR', 'TXUR', 'NYUR', 'FLUR']

# 2. Create an empty list. We will use this to store a "row" of data for each state.
rows = []

# 3. Loop through each ID to fetch and extract its metadata
for series_id in series_ids:
```

```
    # Fetch the metadata (returns a pandas Series with all attributes)
    info = fred.get_series_info(series_id)

    # Extract only the fields we want, format them as a dictionary,
    # and append that dictionary to our 'rows' list.
    rows.append({
        "series_id":            info["id"],
        "title":                info["title"],
        "units":                info["units"],
        "frequency":            info["frequency"],
        "seasonal_adjustment":  info["seasonal_adjustment"],
        "observation_start":    info["observation_start"],
        "observation_end":      info["observation_end"]
    })

# 4. Convert our list of dictionaries directly into a pandas DataFrame
df_info = pd.DataFrame(rows)
```

```
df_info.head()
```

	series_id	title	units	frequency	seasonal_adjustment	observation_start	observation_end
0	UNRATE	Unemployment Rate	Percent	Monthly	Seasonally Adjusted	1948-01-01	2026-03-01
1	CAUR	Unemployment Rate in California	Percent	Monthly	Seasonally Adjusted	1976-01-01	2026-01-01
2	TXUR	Unemployment Rate in Texas	Percent	Monthly	Seasonally Adjusted	1976-01-01	2026-01-01
3	NYUR	Unemployment Rate in New York	Percent	Monthly	Seasonally Adjusted	1976-01-01	2026-01-01
4	FLUR	Unemployment Rate in Florida	Percent	Monthly	Seasonally Adjusted	1976-01-01	2026-01-01

Raising Errors

We have previously discussed the concept of defensive programming - anticipating potential issues and checking inputs before they cause problems. So far we've seen how to use the `return` statement to exit a function early, but not how to raise Errors. This section introduces the "why" and "how" of raising built-in Python errors.

Why raise Errors?

When you write a function that others will use, you want it to fail loudly and clearly rather than silently producing wrong results or a cryptic error message.

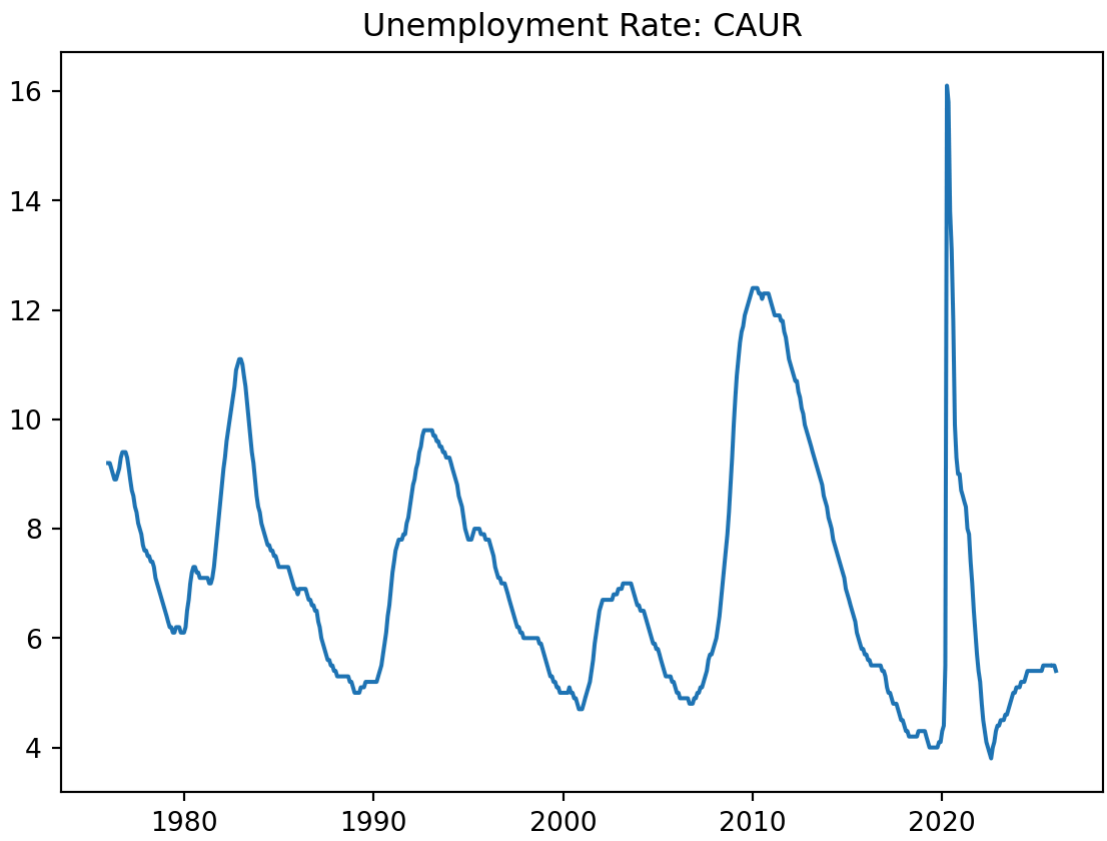
Consider a function that plots an unemployment series:

```
def plot_unemployment(df, series_id):
    plt.plot(df.index, df[series_id])
```

```
    plt.title(f"Unemployment Rate: {series_id}")
    plt.show()
```

This works fine when used correctly:

```
plot_unemployment(df_unemployment, "CAUR")
```



But what happens if a user makes a mistake?

```
plot_unemployment(df_unemployment, "California")
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/base.py:36
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.g

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.g

KeyError: 'California'

The above exception was the direct cause of the following exception:
```

```
KeyError                                  Traceback (most recent call last)
Cell In[82], line 1
----> 1 plot_unemployment(df_unemployment, "California")

Cell In[80], line 2, in plot_unemployment(df, series_id)
      1 def plot_unemployment(df, series_id):
----> 2     plt.plot(df.index, df[series_id])
      3     plt.title(f"Unemployment Rate: {series_id}")
      4     plt.show()

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/frame.py:4378, in
   4376 if self.columns.nlevels > 1:
   4377     return self._getitem_multilevel(key)
-> 4378 indexer = self.columns.get_loc(key)
   4379 if is_integer(indexer):
   4380     indexer = [indexer]

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/base.py:36
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'California'
```

The function crashes with a `KeyError` - which tells the user little about what went wrong or how to fix it. They have to go look at the source code to figure out what values are actually valid. When Python encounters an error, it prints a traceback - a record of every function call that led to the crash, from your code all the way down to the line where it actually failed. The traceback above is also quite unhelpful for a user, the error originates deep inside pandas' internal source code, code the user has never seen and doesn't need to understand.

If some error occurs when the function is called what we want instead is a helpful message explaining what went wrong *and* how to fix it. One approach we have already used is to check the input, print a message, and return early:

```
def plot_unemployment(df, series_id):

    valid_series = df.columns.tolist()                                    (1)
    if series_id not in valid_series:                                     (2)
        print(f"'{series_id}' is not a valid series. Choose from: {valid_series}.")
        return
```

```
    plt.plot(df.index, df[series_id])
    plt.title(f"Unemployment Rate: {series_id}")
    plt.show()
```

This is good, but it has a drawback: the function silently returns `None` and execution continues. Note that while the rest of the code *inside the function* does not run after return, any code *outside the function* will keep running as if nothing went wrong. If this function is part of a larger pipeline, the rest of the code will keep running even though something went wrong - potentially causing confusing errors further down, or worse, producing incorrect results without any warning.

The proper way to handle this is to **raise an error**. Unlike `return`, raising an error stops execution completely - not just inside the function, but also any code that called the function, and any code after that. Python will halt immediately and display the error message, so nothing else runs until the problem is fixed.

How to raise Errors

The syntax is straightforward:

```
raise ErrorType("your message here")
```

- The error message is what the user will see, so it should be clear and informative.
- `ErrorType` is the kind of error you want to raise. Python has many built-in error types — the two you will use most often are:
  - `ValueError`: the input is the right type but an invalid value, for example passing "California" when only series IDs are accepted
  - `TypeError`: the input is the wrong type entirely, for example passing an integer where a string is expected
  - For a broader overview of common Python error types and when they occur, see the Appendix.

Note: You cannot invent your own error type name (line `raise WrongInputError(...)`) you have to choose from Python's built-in types. Technically, which one you choose does not affect how the error behaves: any error type stops execution in exactly the same way. However, the type carries meaning, so it is good practice to choose one that best describes the nature of the problem. When in doubt, `ValueError` is almost always the right choice for input validation. It is possible to define custom error types in Python (such as `WrongInputError()`), but that is beyond the scope of this course.

Here is our function with a proper error:

```
def plot_unemployment(df, series_id):

    valid_series = df.columns.tolist()
    if series_id not in valid_series:
        raise ValueError(f"'{series_id}' is not a valid series. Choose from: {valid_series} 1

    plt.plot(df.index, df[series_id])
    plt.title(f"Unemployment Rate: {series_id}")
    plt.show()
```

Now the same mistake produces:

```
plot_unemployment(df_unemployment, "California")
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[85], line 1
----> 1 plot_unemployment(df_unemployment, "California")

Cell In[84], line 5, in plot_unemployment(df, series_id)
      3 valid_series = df.columns.tolist()
      4 if series_id not in valid_series:
----> 5     raise ValueError(f"'{series_id}' is not a valid series. Choose from: {valid_series}.
      7 plt.plot(df.index, df[series_id])
      8 plt.title(f"Unemployment Rate: {series_id}")

ValueError: 'California' is not a valid series. Choose from: ['UNRATE', 'CAUR', 'TXUR', 'NYUR',
```

When we raise an error ourselves, two things improve. First, the message is informative and tells the user exactly what went wrong *and* how to fix it. Second, the traceback becomes much shorter and points directly to the line in our code where the error was raised. This is useful not just for the end user, but also for your future self or collaborators: when something breaks, a clean traceback pointing to a meaningful error in your own code is much easier to debug than one that disappears into the internals of a third-party library.

> When your code is organized across modules and classes, the traceback follows the full call chain. If you import a class from a module, call one of its methods, and an error is raised inside that method, the traceback will show the file where the method lives, the method name, the specific line where the error was raised, and the line in your code where you called the method. This makes it easy to trace exactly where something went wrong, no matter how your code is organized.

It is good practice to raise errors as early as possible in the function, so that mistakes are caught immediately rather than partway through execution.

Input validation

What we have seen so far is an example of input validation — the general practice of checking that the inputs to a function are valid before doing any actual work. Raising an informative error when something is wrong is the standard way to handle invalid inputs in Python.

There are three common things you might want to validate:

- **Type** — the input is the wrong type entirely. For example, a function expects a string but receives an integer, or expects a DataFrame but receives a list. Type errors are often the result of a simple mistake, like passing arguments in the wrong order.
- **Value** — the input is the right type, but not a valid value. This includes passing a string that doesn't match any of the accepted options, a year that is not available in the data, or a number that falls outside the range the function can handle.
- **Compatibility** — each argument is valid on its own, but two or more arguments conflict with each other. For example, a function has two arguments that represent alternative modes of output — providing both at the same time is ambiguous, so the function should catch this and ask the user to choose one.

The goal is to think about what could plausibly go wrong given how your function will be used, and to catch those cases early with a clear and informative error message.

We will now look at concrete examples of each type of input validation using the `plot_unemployment` function below. This function takes a DataFrame of unemployment series, a list of series IDs to plot, and an optional start and end date. It plots each requested series as a separate line on the same figure, and automatically uses the full available date range if no start or end date is provided.

```
def plot_unemployment(df, series_to_plot, start_date=None, end_date=None):
    """
    Takes a DataFrame and a list of Series IDs, and plots them
    """
    # 1. Filter the DataFrame to keep only the requested columns
    df_filtered = df[series_to_plot]                                        (1)

    # 2. Figure out the actual dates to use in our chart title and slicer
    if start_date is None:                                                  (2)
        start_slice = df_filtered.dropna().index[0]
        title_start = start_slice.strftime('%Y-%m')
    else:
        start_slice = start_date
        title_start = start_date

    if end_date is None:
        end_slice = None
        title_end = df_filtered.index[-1].strftime('%Y-%m')
    else:
        end_slice = end_date
        title_end = end_date

    # 3. Filter the rows to keep only the data within our dynamic bounds
    df_filtered = df_filtered.loc[start_slice:end_slice]                    (3)

    # 4. Create the plot
    plt.figure(figsize=(10, 6))                                             (4)

    for series_id in series_to_plot:                                        (5)
        plt.plot(df_filtered.index, df_filtered[series_id], label=series_id)

    # 5. Add standard chart labels
    plt.title(f'Unemployment Rates ({title_start} to {title_end})')         (6)
    plt.ylabel('Unemployment Rate (%)')
    plt.grid(alpha=0.5)
    plt.legend()
    plt.show()
```

Validating input types

We want to make sure that the inputs to a function are of the expected type. This is useful because Python does not enforce types — you can pass any value to any argument, and the function will only crash when it actually

tries to use it in a way that doesn't work for that type. For example, a function that expects a list might receive a string, or a function that expects an integer might receive a float.

The standard way to check types in Python is with `isinstance()`:

```
isinstance(value, expected_type)  # returns True or False
```

For example:

```
isinstance("CAUR", str)   # True
isinstance(2025, int)     # True
isinstance(2025, str)     # False
```

```
False
```

You can pass a tuple of types to `isinstance()` and it returns `True` if the value matches *any* of them:

```
isinstance(2025, (int, float))  # True — matches either int or float
isinstance("2025", (int, float)) # False
```

```
False
```

This is useful when a function should accept multiple types for the same argument.

Applying this to our function. The `series_to_plot` argument expects a list. If a user wants to plot only one series and passes a single string instead, a natural mistake, the function will not crash immediately, but when the loop iterates over `series_to_plot` it will loop over the individual characters of the string ("C", "A", "U", "R") rather than over series IDs within a list. The error the user eventually gets will be cryptic and give no indication that the real problem was the type of the input.

```
plot_unemployment(df_unemployment, 'CAUR')
```

```
---------------------------------------------------------------------------
DateParseError                            Traceback (most recent call last)
File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/datetimes.
    968 try:
--> 969     parsed, reso = self._parse_with_reso(key)
    970 except ValueError as err:

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/datetimes.
    924 def _parse_with_reso(self, label: str) -> tuple[Timestamp, Resolution]:
--> 925     parsed, reso = super()._parse_with_reso(label)
    927     parsed = Timestamp(parsed)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/datetimeli
    410     label = str(label)
--> 412 parsed, reso_str = parsing.parse_datetime_string_with_reso(label, freqstr)
    413 reso = Resolution.from_attrname(reso_str)

File pandas/_libs/tslibs/parsing.pyx:428, in pandas._libs.tslibs.parsing.parse_datetime_string_w
```

```
File pandas/_libs/tslibs/parsing.pyx:652, in pandas._libs.tslibs.parsing.dateutil_parse()

DateParseError: Unknown datetime string format, unable to parse: C

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[89], line 1
----> 1 plot_unemployment(df_unemployment, 'CAUR')

Cell In[86], line 30, in plot_unemployment(df, series_to_plot, start_date, end_date)
     27 plt.figure(figsize=(10, 6))                            #<4>
     29 for series_id in series_to_plot:                       #<5>
---> 30     plt.plot(df_filtered.index, df_filtered[series_id], label=series_id)  #<5>
     32 # 5. Add standard chart labels
     33 plt.title(f'Unemployment Rates ({title_start} to {title_end})')  #<6>

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/series.py:959, in
    954     key = unpack_1tuple(key)
    956 elif key_is_scalar:
    957     # Note: GH#50617 in 3.0 we changed int key to always be treated as
    958     #  a label, matching DataFrame behavior.
--> 959     return self._get_value(key)
    961 # Convert generator to list before going through hashable part
    962 # (We will iterate through the generator there to check for slices)
    963 if is_iterator(key):

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/series.py:1046, in
   1043     return self._values[label]
   1045 # Similar to Index.get_value, but we do not fall back to positional
-> 1046 loc = self.index.get_loc(label)
   1048 if is_integer(loc):
   1049     return self._values[loc]

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/datetimes.
    969     parsed, reso = self._parse_with_reso(key)
    970 except ValueError as err:
--> 971     raise KeyError(key) from err
    972 self._disallow_mismatched_indexing(parsed)
    974 if self._can_partial_date_slice(reso):

KeyError: 'C'
```

```
<Figure size 960x576 with 0 Axes>
```

We can catch this early with an `isinstance()` check:

```
def plot_unemployment(df, series_to_plot, start_date=None, end_date=None):

    if not isinstance(series_to_plot, list):
        raise TypeError(f"series_to_plot must be a list, got {type(series_to_plot).__name__} i
```

```
    ...
```

> Note the use of `type(series_to_plot).__name__` in the error message — this extracts the name of the type the user actually passed, which makes the message more informative than just saying "wrong
>
> it the message would look like: `TypeError: series_to_plot must be a list, got <class 'str'> instead.`

Similarly we can add type checks for all four arguments. `df` is checked first since everything else depends on it being a DataFrame. `series_to_plot must` be a list, as passing a string would cause the loop to iterate over individual characters rather than series IDs. For `start_date` and `end_date`, note the `is not None and` before the type check - since these are optional arguments, we only validate the type if the user actually provided a value. Checking `None` with `isinstance()` would incorrectly raise an error when the user simply omits the argument.

```
def plot_unemployment(df, series_to_plot, start_date=None, end_date=None):

    # Validating input types
    if not isinstance(df, pd.DataFrame):
        raise TypeError(f"df must be a pandas DataFrame, got {type(df).__name__} instead.")

    if not isinstance(series_to_plot, list):
        raise TypeError(f"series_to_plot must be a list, got {type(series_to_plot).__name__} i

    if start_date is not None and not isinstance(start_date, str):
        raise TypeError(f"start_date must be a string, got {type(start_date).__name__} instead

    if end_date is not None and not isinstance(end_date, str):
        raise TypeError(f"end_date must be a string, got {type(end_date).__name__} instead.")
    ...
```

▶ Show full code

If the user passes a string, they now get a much more informative error:

```
plot_unemployment(df_unemployment, 'CAUR')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[91], line 1
----> 1 plot_unemployment(df_unemployment, 'CAUR')

Cell In[90], line 15, in plot_unemployment(df, series_to_plot, start_date, end_date)
     12     raise TypeError(f"df must be a pandas DataFrame, got {type(df).__name__} instead.")
     14 if not isinstance(series_to_plot, list):
```

```
---> 15     raise TypeError(f"series_to_plot must be a list, got {type(series_to_plot).__name__}
     17 if start_date is not None and not isinstance(start_date, str):
     18     raise TypeError(f"start_date must be a string, got {type(start_date).__name__} inste

TypeError: series_to_plot must be a list, got str instead.
```

Validating input values

We also want to make sure that the inputs are not just the right type, but also a valid value. This includes checking that a string matches one of the accepted options, that a value actually exists in the data, that a number falls within an acceptable range,… The key difference from type validation is that the input is the right type — it just doesn't make sense given what the function expects.

Applying this to our function. There are two natural value checks here. First, each series ID in `series_to_plot` should actually exist as a column in the DataFrame — a user might mistype a series ID or pass one that is not in the data. Second, if a start or end date is provided in the correct type (string) we need to check whether the string actually represents a real date? (e.g., "2020-01-01" is valid, but "Hello" or "2020-13-45" are not) and it should fall within the range of dates actually available in the data.

```
def plot_unemployment(df, series_to_plot, start_date=None, end_date=None):

    ...

    # Validating input values

    # 1. Check if series IDs exist
    valid_series = df.columns.tolist()
    invalid = [s for s in series_to_plot if s not in valid_series]                    1
    if invalid:
        raise ValueError(f"The following series were not found in the DataFrame: {invalid}. Av

    # 2. Check if dates are valid and within range
    earliest = df.index.min()
    latest = df.index.max()

    if start_date is not None:
        start_ts = pd.to_datetime(start_date, errors='coerce')                        2
        if pd.isna(start_ts):                                                         3
            raise ValueError(f"start_date '{start_date}' could not be parsed. Please use a val
        if start_ts < earliest:                                                       4
            raise ValueError(f"start_date '{start_date}' is before the earliest available date

    if end_date is not None:
        end_ts = pd.to_datetime(end_date, errors='coerce')
        if pd.isna(end_ts):
            raise ValueError(f"end_date '{end_date}' could not be parsed. Please use a valid d
        if end_ts > latest:
            raise ValueError(f"end_date '{end_date}' is after the latest available date {lates
```

```
    ...
```

```
plot_unemployment(df_unemployment, series_to_plot=['CAUR'], start_date='2005-01-99')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[93], line 1
----> 1 plot_unemployment(df_unemployment, series_to_plot=['CAUR'], start_date='2005-01-99')

Cell In[92], line 37, in plot_unemployment(df, series_to_plot, start_date, end_date)
     35 start_ts = pd.to_datetime(start_date, errors='coerce')
     36 if pd.isna(start_ts):
---> 37     raise ValueError(f"start_date '{start_date}' could not be parsed. Please use a valid
     38 if start_ts < earliest:
     39     raise ValueError(f"start_date '{start_date}' is before the earliest available date {

ValueError: start_date '2005-01-99' could not be parsed. Please use a valid date format like 'YY
```

▶ Show full code

Validating compatibility of inputs

We also want to make sure that the combination of arguments provided makes sense, even if each argument is valid on its own. This is relevant when a function has arguments that interact with each other — for example two arguments that represent alternative options, or a start and end that must be in the right order relative to each other.

Applying this to our function. The natural compatibility check here is that if both `start_date` and `end_date` are provided, `start_date` must come before `end_date`. Each date might be individually valid - both within the

```
def plot_unemployment(df, series_to_plot, start_date=None, end_date=None):

    ...
    # Validating argument compatibility
    if start_date is not None and end_date is not None and pd.Timestamp(start_date) >= pd.Time
        raise ValueError(f"start_date must be before end_date, got start_date='{start_date}' a

    ...
```

```
plot_unemployment(df_unemployment, series_to_plot=['CAUR'], start_date='2010-01-01', end_date=
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[95], line 1
----> 1 plot_unemployment(df_unemployment, series_to_plot=['CAUR'], start_date='2010-01-01', end

Cell In[94], line 51, in plot_unemployment(df, series_to_plot, start_date, end_date)
     49 # Validating compatibility of inputs
     50 if start_date is not None and end_date is not None and pd.Timestamp(start_date) > pd.Tim
---> 51     raise ValueError(f"start_date must be before end_date, got start_date='{start_date}'
     53 # ========================================
     54
     55
     56 # 1. Filter the DataFrame to keep only the requested columns
     57 df_filtered = df[series_to_plot]

ValueError: start_date must be before end_date, got start_date='2010-01-01' and end_date='2005-0
```

▶ Show full code

What makes a good error message

A good error message does three things:

- **Says what went wrong** — be specific about which argument or condition caused the problem
- **Shows the invalid value** — include the actual value the user passed, using an f-string
- **Says what is valid instead** — tell the user what they should pass, whether that is a list of valid options or a valid range

For example, compare these two messages for the same error:

- Bad: `"invalid input"`
- Good: `"start_date '1800-01-01' is before the earliest available date 1948-01-01."`

Write error messages as if you are explaining the problem to someone who has never seen your code. Be informative and to the point — you don't need to explain your implementation, just tell the user what they need to know to fix the problem.

Type Hints

Type hints are a way to annotate function signatures to indicate what type each argument expects and what the function returns. They do not affect how the code runs - Python does not enforce them - but they make your code more readable and help users understand how to use a function without having to read through its full implementation.

The syntax is straightforward. For arguments, add `: type` after the argument name. For the return value, add `-> type` after the closing parenthesis:

```
def plot_unemployment(df: pd.DataFrame, series_to_plot: list, start_date: str | None = None, e
```

```
    ...
```

For optional arguments that default to `None`, use `type | None` to indicate that the argument can be either the expected type or `None`. The `-> None` at the end indicates that the function does not return anything.

different purposes. Type hints are documentation - they tell the user what to pass, but Python will not stop them from passing the wrong type. Input validation with `isinstance()` is what actually catches mistakes at runtime and raises an informative error. In practice you want both: type hints so users know what is expected, and validation so the function fails clearly if they get it wrong.

Appendix

Below is an overview of the most common error types you will encounter in Python:

1. `SyntaxError`: This happens when Python's grammar rules are broken. The interpreter cannot even begin to read or execute the code because the structure is fundamentally invalid.

Example `SyntaxError`

```
gdp_growth = 2.5

# This will raise a SyntaxError because the colon is missing
if gdp_growth > 0
    print("The economy is expanding.")
```

```
  Cell In[96], line 4
    if gdp_growth > 0
                     ^
SyntaxError: expected ':'
```

2. `TypeError`: This occurs when you try to perform an operation on an object of the wrong data type. It is the classic "apples and oranges" problem. In pandas this frequently happens when you try to apply a mathematical function to a column that contains strings, or when you pass the wrong type of argument into a Pandas method.

Example `TypeError`

```
inflation_rate = 0.05

# This will raise a TypeError. You must convert 0.05 to a string using str() first.
message = "The current inflation rate is " + inflation_rate
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[97], line 4
      1 inflation_rate = 0.05
```

```
      3 # This will raise a TypeError. You must convert 0.05 to a string using str() first.
----> 4 message = "The current inflation rate is " + inflation_rate

TypeError: can only concatenate str (not "float") to str
```

```
# Pandas context: Trying to calculate the mean of a string column
df = pd.DataFrame({'Country': ['USA', 'UK', 'JPN']})
average_country = df['Country'].mean()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[98], line 3
      1 # Pandas context: Trying to calculate the mean of a string column
      2 df = pd.DataFrame({'Country': ['USA', 'UK', 'JPN']})
----> 3 average_country = df['Country'].mean()

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/util/_decorators.py:336, in dep
    330 if len(args) > num_allow_args:
    331     warnings.warn(
    332         msg.format(arguments=_format_argument_list(allow_args)),
    333         klass,
    334         stacklevel=find_stack_level(),
    335     )
--> 336 return func(*args, **kwargs)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/series.py:8113, in Series
   8063 @deprecate_nonkeyword_arguments(Pandas4Warning, allowed_args=["self"], name="mean")
   8064 def mean(
   8065     self,
   (...)   8069     **kwargs,
   8070 ) -> Any:
   8071     """
   8072     Return the mean of the values over the requested axis.
   8073
   (...)   8111     2.0
   8112     """
-> 8113     return NDFrame.mean(
   8114         self, axis=axis, skipna=skipna, numeric_only=numeric_only, **kwargs
   8115     )

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/generic.py:11831, in NDFra
  11823 def mean(
  11824     self,
  11825     *,
   (...)  11829     **kwargs,
  11830 ) -> Series | float:
> 11831     return self._stat_function(
  11832         "mean", nanops.nanmean, axis, skipna, numeric_only, **kwargs
  11833     )

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/generic.py:11785, in NDFra
  11781 nv.validate_func(name, (), kwargs)
  11783 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 11785 return self._reduce(
  11786     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_only
  11787 )

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/series.py:7480, in Series
```

```
   7476     self._get_axis_number(axis)
   7478 if isinstance(delegate, ExtensionArray):
   7479     # dispatch to ExtensionArray interface
-> 7480     result = delegate._reduce(name, skipna=skipna, **kwds)
   7482 else:
   7483     # dispatch to numpy arrays
   7484     if numeric_only and self.dtype.kind not in "iufcb":
   7485         # i.e. not is_numeric_dtype(self.dtype)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/arrays/string_.py:967, in
    964         return self._from_sequence([result], dtype=self.dtype)
    965     return result
--> 967 raise TypeError(f"Cannot perform reduction '{name}' with string dtype")

TypeError: Cannot perform reduction 'mean' with string dtype
```

3. `AttributeError`: This happens when you try to use a method or property that does not belong to the specific type of object you are working with.

Example `AttributeError`

```python
interest_rates = [0.02, 0.025, 0.03, 0.035]

# This will raise an AttributeError. Lists don't have a .mean() method.
# You would need sum(interest_rates) / len(interest_rates) or a numpy array.
average_rate = interest_rates.mean()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[99], line 5
      1 interest_rates = [0.02, 0.025, 0.03, 0.035]
      3 # This will raise an AttributeError. Lists don't have a .mean() method.
      4 # You would need sum(interest_rates) / len(interest_rates) or a numpy array.
----> 5 average_rate = interest_rates.mean()

AttributeError: 'list' object has no attribute 'mean'
```

4. `IndexError`: This occurs when you try to access an item in a sequence (like a list, tuple, or string) using an index number that is out of bounds. In pandas it happens when using integer-position based indexing (specifically `.iloc[]`) and requesting a row or column position that does not exist in the DataFrame.

Example `IndexError`

```python
quarters = ["Q1", "Q2", "Q3"]

# This will raise an IndexError. The valid indices are 0, 1, and 2.
current_quarter = quarters[3]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[100], line 4
      1 quarters = ["Q1", "Q2", "Q3"]
      3 # This will raise an IndexError. The valid indices are 0, 1, and 2.
----> 4 current_quarter = quarters[3]

IndexError: list index out of range
```

```
# Pandas context: Asking for row 10 in a 3-row DataFrame
df = pd.DataFrame({'Year': [2020, 2021, 2022]})
tenth_year_data = df.iloc[9]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[101], line 3
      1 # Pandas context: Asking for row 10 in a 3-row DataFrame
      2 df = pd.DataFrame({'Year': [2020, 2021, 2022]})
----> 3 tenth_year_data = df.iloc[9]

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexing.py:1207, in _Loc
   1205 maybe_callable = com.apply_if_callable(key, self.obj)
   1206 maybe_callable = self._raise_callable_usage(key, maybe_callable)
-> 1207 return self._getitem_axis(maybe_callable, axis=axis)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexing.py:1773, in _iLo
   1770     raise TypeError("Cannot index by location index with a non-integer key")
   1772 # validate the location
-> 1773 self._validate_integer(key, axis)
   1775 return self.obj._ixs(key, axis=axis)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexing.py:1706, in _iLo
   1704 len_axis = len(self.obj._get_axis(axis))
   1705 if key >= len_axis or key < -len_axis:
-> 1706     raise IndexError("single positional indexer is out-of-bounds")

IndexError: single positional indexer is out-of-bounds
```

5. `ValueError`: This happens when a function receives an argument of the correct type (e.g., a string), but the value inside that string is inappropriate for the operation. In pandas it also happens for for “shape mismatches.” It occurs when you try to align data that does not physically fit together, such as assigning a list of five values to a DataFrame column that only has three rows.

Example `ValueError`

```
user_input = "2020 Q1"

# This will raise a ValueError. Python knows how to convert strings to integers,
# but it doesn't know what integer " Q1" translates to.
year = int(user_input)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[102], line 5
      1 user_input = "2020 Q1"
      3 # This will raise a ValueError. Python knows how to convert strings to integers,
      4 # but it doesn't know what integer " Q1" translates to.
----> 5 year = int(user_input)

ValueError: invalid literal for int() with base 10: '2020 Q1'
```

```python
# Pandas context: Shape mismatch during column assignment
df = pd.DataFrame({'Year': [2020, 2021, 2022]})
# Fails because the list has 4 items, but the DataFrame only has 3 rows
df['Unemployment'] = [6.7, 5.3, 3.6, 3.8]
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[103], line 4
      2 df = pd.DataFrame({'Year': [2020, 2021, 2022]})
      3 # Fails because the list has 4 items, but the DataFrame only has 3 rows
----> 4 df['Unemployment'] = [6.7, 5.3, 3.6, 3.8]

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/frame.py:4672, in DataFra
   4669     self._setitem_array([key], value)
   4670 else:
   4671     # set column
-> 4672     self._set_item(key, value)

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/frame.py:4872, in DataFra
   4862 def _set_item(self, key, value) -> None:
   4863     """
   4864     Add series to DataFrame in specified column.
   4865
   (...)   4870     ensure homogeneity.
   4871     """
-> 4872     value, refs = self._sanitize_column(value)
   4874     if (
   4875         key in self.columns
   4876         and value.ndim == 1
   4877         and not isinstance(value.dtype, ExtensionDtype)
   4878     ):
   4879         # broadcast across multiple columns if necessary
   4880         if not self.columns.is_unique or isinstance(self.columns, MultiIndex):

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/frame.py:5742, in DataFra
   5739     return _reindex_for_setitem(value, self.index)
   5741 if is_list_like(value):
-> 5742     com.require_length_match(value, self.index)
   5743 return sanitize_array(value, self.index, copy=True, allow_2d=True), None

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/common.py:601, in require
    597 """
    598 Check the length of data matches the length of the index.
    599 """
    600 if len(data) != len(index):
--> 601     raise ValueError(
    602         "Length of values "
    603         f"({len(data)}) "
```

```
    604         "does not match length of index "
    605         f"({len(index)})"
    606     )

ValueError: Length of values (4) does not match length of index (3)
```

6. `KeyError`: This happens when you ask a dictionary to return a value for a key that has not been defined in its records. In pandas it happens when you try to select or operate on a column name that does not exist (often due to typos, case sensitivity, or hidden whitespace).

Example `KeyError`

```python
# Asking for a missing dictionary key
gdp_data = {"USA": 25.4, "CHN": 17.9}
germany_gdp = gdp_data["DEU"]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[104], line 3
      1 # Asking for a missing dictionary key
      2 gdp_data = {"USA": 25.4, "CHN": 17.9}
----> 3 germany_gdp = gdp_data["DEU"]

KeyError: 'DEU'
```

```python
data = {
    'year': [2020, 2021, 2022],
    'gdp_billions': [21060, 23315, 25462]
}
df = pd.DataFrame(data)

# This will raise a KeyError because the column is named 'gdp_billions', not 'gdp'
gdp_column = df['gdp']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/base.py:3641, in 
   3640 try:
-> 3641     return self._engine.get_loc(casted_key)
   3642 except KeyError as err:

File pandas/_libs/index.pyx:168, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:197, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7668, in pandas._libs.hashtable.PyObjectHashTable.get_item

File pandas/_libs/hashtable_class_helper.pxi:7676, in pandas._libs.hashtable.PyObjectHashTable.get_item

KeyError: 'gdp'

The above exception was the direct cause of the following exception:
```

```
KeyError                                  Traceback (most recent call last)
Cell In[105], line 8
      5 df = pd.DataFrame(data)
      7 # This will raise a KeyError because the column is named 'gdp_billions', not 'gdp'
----> 8 gdp_column = df['gdp']

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/frame.py:4378, in DataFra
   4376 if self.columns.nlevels > 1:
   4377     return self._getitem_multilevel(key)
-> 4378 indexer = self.columns.get_loc(key)
   4379 if is_integer(indexer):
   4380     indexer = [indexer]

File ~/Desktop/Intro_Python_FB/.venv/lib/python3.14/site-packages/pandas/core/indexes/base.py:3648, in
   3643     if isinstance(casted_key, slice) or (
   3644         isinstance(casted_key, abc.Iterable)
   3645         and any(isinstance(x, slice) for x in casted_key)
   3646     ):
   3647         raise InvalidIndexError(key) from err
-> 3648     raise KeyError(key) from err
   3649 except TypeError:
   3650     # If we have a listlike key, _check_indexing_error will raise
   3651     #  InvalidIndexError. Otherwise we fall through and re-raise
   3652     #  the TypeError.
   3653     self._check_indexing_error(key)

KeyError: 'gdp'
```

Some other common Python errors:

- `ZeroDivisionError`: Raised when the second argument of a division or modulo operation is zero. In economics, this frequently happens when calculating percentage changes, growth rates, or per-capita metrics where the base year or population data is unexpectedly recorded as zero.
- `FileNotFoundError`: Triggers when you attempt to open a file or directory that does not exist. This is the classic data-loading error, usually caused by a typo in the file path or being in the wrong working directory
- `ModuleNotFoundError`: Occurs when you try to import a library that Python cannot find. Usually occurs when the library name is misspelled or the package isn't installed in the current environment.
- …